

AI-Based Face Extraction and Clustering Pipeline

Intricate architecture description with two-stage clustering, unified PostgreSQL + pgvector store, and four isolated workflows.

Version: Refined Architecture

Audience: System architects, backend engineers, and ML infrastructure teams.

This document is an engineering reference, capturing implementation-level details suitable for production deployment.

Table of Contents

AI-Based Face Extraction and Clustering Pipeline	1
Table of Contents	2
1. Executive Summary	4
2. System Architecture Overview	4
3. Workflow 1 – Ingestion (Batch Processing)	4
3.1 Input and Triggers	4
3.2 AI Worker Responsibilities	4
3.3 Reliability Concerns	5
4. Workflow 2 – Two-Stage Clustering & Storage	5
4.1 Unified Vector Schema	5
4.2 Partitioning Strategy	5
4.3 Real-Time Clustering Worker	5
4.4 Global Merge Worker	5
4.5 Metrics and Tuning	6
5. Workflow 3 – Data Retrieval & Analytics	6
5.1 API Surface	6
5.2 Caching and Read Replicas	6
6. Workflow 4 – Manual Face Capture	6
6.1 Use-Case Routing	6
7. Deployment & Operations	6
7.1 Monitoring and Alerting	6
8. Architecture Diagram	7
Figure 1 – Refined Face Pipeline Architecture	8

1. Executive Summary

The pipeline is designed to support high-volume event photography and enterprise biometric workflows by separating ingestion, clustering, retrieval, and manual capture into four independent, asynchronous workflows. Each workflow communicates via durable Pub/Sub topics and shares a unified master database with vector capabilities.

The core innovation is a two-stage clustering strategy. A real-time clustering worker operates exclusively on a Hot partition backed by an HNSW index, delivering sub-100 ms person assignment even when the historical corpus spans hundreds of millions of faces. An asynchronous global merge worker periodically reconciles local identities against a Cold historical partition, ensuring global uniqueness without impacting live SLAs.

This design avoids cross-database synchronization issues by storing metadata and embeddings together in PostgreSQL with pgvector, while still allowing FAISS-style indexing strategies such as IVFFlat to be emulated through pgvector indexes and partitioning.

2. System Architecture Overview

At a high level, the system accepts images from photographer devices and enterprise entry points, converts faces into high-dimensional embeddings, clusters these embeddings into unique persons, and exposes analytics and search APIs. The four workflows are:

- Workflow 1 – Ingestion: Automatic batch ingestion of images, face detection, embedding extraction, and demographics classification.
- Workflow 2 – Two-Stage Clustering & Storage: Real-time event-local clustering plus asynchronous global identity reconciliation backed by a unified pgvector store.
- Workflow 3 – Data Retrieval & Analytics: Read-only dashboards and APIs for statistics, reverse face search, and galleries.
- Workflow 4 – Manual Face Capture: Real-time manual uploads for access control, attendance, and security scenarios, isolated from event ingestion.

3. Workflow 1 – Ingestion (Batch Processing)

Workflow 1 orchestrates the full lifecycle of image ingestion, from upload through to embedding publication. It is optimized for GPU utilization and resilience to bursty traffic.

3.1 Input and Triggers

Images are uploaded from photographer devices to a cloud storage bucket. Object creation events trigger a lightweight Cloud Function or similar serverless component, which groups new images into batches and publishes batch descriptors onto the Raw Images Pub/Sub topic. Batching reduces per-image overhead and increases GPU throughput.

3.2 AI Worker Responsibilities

Each AI worker pod mounts the relevant model weights and performs the following steps for each batch:

1. Load image metadata (paths, event identifiers, photographer IDs).
2. Perform face detection using a model such as RetinaFace, generating bounding boxes and keypoints.

3. Align faces (e.g., eye-centering and cropping) to improve embedding quality.
4. Run the InsightFace embedding model in FP16 to produce 512-dimensional normalized vectors.
5. Optionally run lightweight heads for age bracket and gender classification.
6. Write temporary records (face_id, image_path, embedding, demographics, detection confidence) into a short-term in-memory index or cache.
7. Emit an Embeddings event to the Embeddings Pub/Sub topic, containing references to embeddings and minimal metadata required by Workflow 2.

3.3 Reliability Concerns

Failed batch processing attempts are retried a bounded number of times. Messages that still fail after retries are moved to a dead-letter queue for investigation, ensuring that Poison Pill images do not block the pipeline. GPU health metrics are continuously monitored, and pods with repeated GPU errors are drained and restarted.

4. Workflow 2 – Two-Stage Clustering & Storage

Workflow 2 is responsible for transforming a stream of raw embeddings into stable, globally unique person identities while maintaining strict latency guarantees for live events.

4.1 Unified Vector Schema

The master faces table typically contains columns such as face_id (UUID), person_id (UUID), event_id, capture_timestamp, photographer_id, age_bucket, gender, quality_score, image_uri, and embedding (vector(512)). Normalization constraints ensure that every embedding row references exactly one person row and that person rows can be enriched with cross-event attributes (e.g., VIP flags).

4.2 Partitioning Strategy

Logical Hot and Cold partitions can be implemented using PostgreSQL declarative partitioning by event_date or ingestion window. HNSW indexes are attached to the Hot partitions to serve low-latency queries, while IVFFlat indexes can be attached to Cold partitions to accelerate approximate search over large historical segments. Maintenance jobs periodically move older rows from Hot to Cold to keep the Hot index inside memory budgets.

4.3 Real-Time Clustering Worker

The real-time worker pulls from the Embeddings queue and processes each face independently. By querying only the Hot partition with a bounded radius in embedding space, it computes a small candidate set of nearest neighbors. If the best neighbor distance lies below a configured threshold and the quality scores are acceptable, the new embedding inherits that neighbor's person_id. Otherwise, a new person row is inserted and its ID is used. This decision is logged for auditing, and the embedding row is updated in a single transaction.

4.4 Global Merge Worker

The global merge worker consumes a separate Global Merge queue. For each record, it runs broader similarity searches that may span multiple partitions, allowing it to catch cross-event duplicates that the event-local clustering could not see. When duplicates are found, it writes a merge record indicating the source and target person_ids, updates all affected face rows, and optionally triggers cache invalidations for downstream services.

4.5 Metrics and Tuning

Metric	Target / Notes
Hot Search Latency (p95)	< 50 ms per query; adjust HNSW ef_search accordingly
Global Merge Throughput	Batch-oriented; acceptable to be minutes behind real time
Duplicate Elimination Rate	30–50% depending on event type and photographer behavior
Index Rebuild Window	Off-peak hours; use rolling index rebuilds for Cold partitions

5. Workflow 3 – Data Retrieval & Analytics

Workflow 3 exposes the clustered data to end users without interfering with ingestion or clustering. It relies on read replicas where possible and uses pagination, caching, and rate limiting to protect the core database.

5.1 API Surface

The API typically exposes routes for live demographics, top photographers, per-person galleries, and reverse face search. Each route is designed to translate naturally into either SQL aggregations or pgvector similarity queries.

5.2 Caching and Read Replicas

Dashboard-style queries are cached for short intervals (tens of seconds) to absorb spikes in dashboard refreshes. Reverse face search calls may bypass cache but are rate-limited and directed to read replicas that mirror the master, ensuring search does not contend with clustering updates.

6. Workflow 4 – Manual Face Capture

Workflow 4 offers a low-latency path for single-face captures from enterprise systems. It runs on a dedicated queue such that even a large event backlog cannot delay access control or attendance decisions.

6.1 Use-Case Routing

After embedding extraction and vector search, the system evaluates the embedding against configured business rules: access-control lists, employee directories, or watch lists. Each decision path publishes a domain-specific event (e.g., EMPLOYEE_ENTRY_VERIFIED, UNKNOWN_PERSON_SEEN, VISITOR_BADGE_CREATED) which downstream services handle independently.

7. Deployment & Operations

The platform is typically deployed on Kubernetes with separate node pools for GPU and CPU workloads. Autoscaling policies are tuned per workflow: GPU pools scale with Raw Images queue depth, clustering workers scale with Embeddings and Global Merge backlogs, and API replicas scale with request rate and latency.

7.1 Monitoring and Alerting

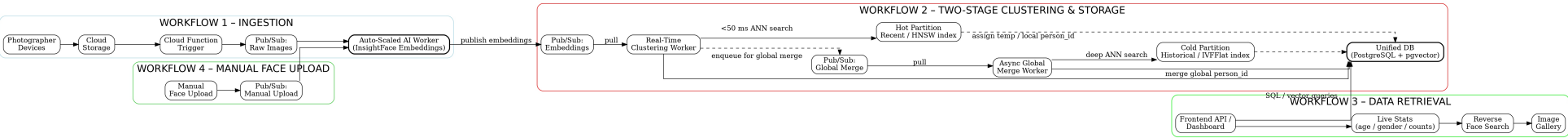
Key metrics include queue depth, worker error rates, GPU utilization, database replication lag, index size, and p95 latencies for embedding, clustering, and query paths. Alert thresholds are defined such that sustained violations of SLAs (for example, Global Merge lagging more than a configured number

of minutes) trigger on-call responses.

8. Architecture Diagram

The complete architecture diagram is rendered on the next page in landscape orientation. It highlights message flows between workflows and the two-stage clustering path through the unified database.

Figure 1 – Refined Face Pipeline Architecture



9. Conclusion

By combining asynchronous messaging, GPU-accelerated embedding extraction, and a two-stage clustering strategy over a unified pgvector-backed store, this architecture delivers both real-time performance and long-term scalability. Each workflow can be evolved independently, enabling the system to adapt to new models, hardware, and enterprise integrations without major redesign.